TITLE: - “AI-Powered Disaster Response & Safety Alert Platform”

Project Overview: -



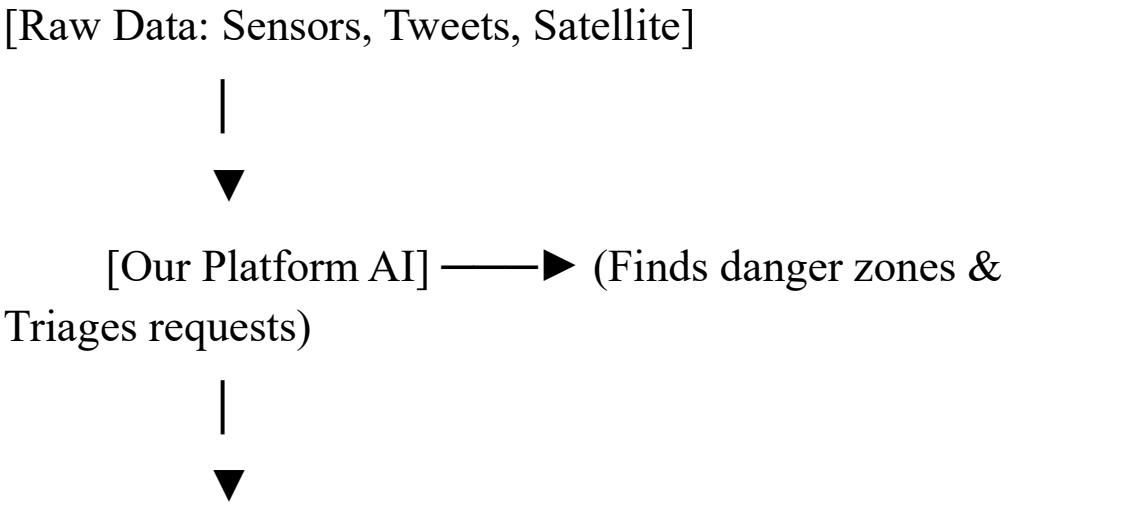
1. The Problem (Why This Matters)

When natural disasters (like floods, earthquakes, or wildfires) strike, every second counts. Unfortunately, current emergency systems face huge problems:

- **Information Overload:** Emergency rooms are flooded with thousands of phone calls and chaotic social media posts, making it hard to see who needs help first.
- **Slow Warnings:** Warning systems often broadcast to entire cities instead of warning the specific neighbourhoods in immediate danger.
- **Communication Breakdown:** Different rescue teams use different systems, making it hard to share data quickly.

2. Our Solution (The Vision)

Our platform is an intelligent, cloud-based system designed to turn chaotic disaster data into fast, life-saving actions. It acts as an **all-in-one emergency brain** that connects citizens, artificial intelligence, and rescue teams together in real-time.



3. How It Works (The 3 Core Pillars)

Step 1: Gathering the Data (Ingestion)

The platform constantly listens to multiple data streams, including:

- Weather and seismic sensors (measuring water levels, earthquakes, etc.).
- Public social media feeds (Twitter/X, Facebook) and text messages (SMS, WhatsApp).
- Satellite and drone imagery of disaster areas.

Step 2: Figuring Out the Situation (AI Processing)

Once data comes in, our AI models do the heavy lifting:

- **Computer Vision:** Looks at satellite pictures to automatically spot flooded roads, broken bridges, or collapsed buildings.
- **Smart Text Reading (NLP):** Scans incoming SOS messages to filter out spam, find where the person is, and judge how badly they are hurt.
- **Predictive Mapping:** Calculates where a fire or flood will spread *before* it actually gets there.

Step 3: Taking Action (Smart Alerts)

Instead of sending a blanket warning, the system draws a digital boundary around the exact danger zone and pushes out alerts:

- **To Citizens:** Sends instant SMS, WhatsApp messages, or loud phone calls with exact evacuation routes to people inside the danger zone.
- **To First Responders:** Gives rescue teams a live map showing them the safest, fastest routes, avoiding blocked roads.

4. Key Features

- **Live Danger Heatmaps:** A visual map for authorities showing changing threat levels in real-time.
- **Automated SOS Triage:** Automatically ranks rescue requests by urgency so responders can save the most critical lives first.

- **Multi-Channel Warnings:** Works across mobile apps, SMS text, automated voice calls, and physical smart sirens.
- **Offline Resilience:** Designed to eventually work via local mesh networks (like Bluetooth/LoRa) if cell towers collapse.

5. Technical Ecosystem (The Tech Stack)

Component	Technology	What It Does
User & Admin Interface	React.js, Flutter, Map box	The apps citizens use and the live map dashboard for admins.
Backend Brain	Node.js, Fast API	Connects all the services and handles system logic.
Data Streams	Apache Kafka, WebSocket's	Feeds live data into the platform without lagging.
AI Modules	Porch, Hugging Face, OpenCV	Powers the image scanning, text reading, and predictions.

Component	Technology	What It Does
Database & Storage	PostgreSQL (Postages), AWS S3	Safely stores user locations, map boundaries, and images.
Notification Engine	Twilio, Firebase (FCM)	Sends the actual text messages, push notifications, a calls.

System Setup:-

System Assembly, Configuration & Setup Guide

This guide provides step-by-step instructions to assemble, configure, and initialize the real-time ecosystem for its very first use. Follow these steps to connect the **User Emergency App**, the **Backend Infrastructure**, and the **Operations Control Centre**.

Project Directory Structure

Before configuring the system, ensure your local workspace matches the updated directory layout:

- /User-app: The citizen-facing mobile/web portal where users trigger SOS alerts and upload images.
- /Backend: The central Node.js, Express, and Socket.IO real-time server core.

- /Controller-app: The React-based Operations Dashboard featuring tactical maps and AI prioritization tools.

Step 1: Initialize System Dependencies

You must install the necessary runtime packages for both the backend core and the dashboard.

1. Initialize the Backend Core

1. Open a terminal and navigate to the backend folder:

cd backend

2. Install the required server dependencies (Express, Socket.IO, etc.):

2. Initialize the Operations Control Centre

1. Open a second terminal window and navigate to the controller dashboard folder:

Bash

cd controller-app

2. Install the React packages (Tailwind CSS, Leaflet, Lucide React, Recharts, and Socket.IO client):

```bash

npm install

**Step 2: Zero-Config Local Database Preparation**

The platform features an automated **Zero-Config Database** engine out of the box.
```

- **First-Use Behavior:** When launched for the first time, database.js automatically creates and populates local JSON storage files.
- **Pre-Seeded Data:** The system will automatically seed default safe-haven shelters, active responder units, and testing accounts. No manual SQL or MongoDB schema configuration is required during your initial local run.

Step 3: Launching the Platform (First-Use Verification)

Due to script safety settings or file-system locking on some machines (such as Dropbox conflicts handled by our custom Vite configurations), execute the exact startup sequences below.

Terminal 1: Spin Up the Backend Server Core

Run the following command in your backend directory to unlock PowerShell execution constraints, locate Node.js, and start the live event loop engine:

PowerShell

Set-Execution Policy -Execution Policy Bypass -Scope Process; $env: Path = 'C:\Program Files\NodeJS;' + $env: Path; npm start

What this does: Boots up server.js on **http://localhost:5000**. This initializes the active citizen registries, GPS proximity math, and the incoming Base64 image handler pipelines.

Terminal 2: Boot up the Operations Dashboard

Run the following command in your controller-app directory to launch the NASA-style tactical map and analytics interface:

PowerShell

Set-Execution Policy -Execution Policy Bypass -Scope Process; $env: path= 'C:\Program Files\NodeJS;' + $env: path; npm run dev

What this does: Boots up the Vite development server on **http://localhost:5173** to display the real-time operational hub.

Step 4: Step-by-Step System Testing & Binding Check

To verify that the real-time cross-channel broadcast system is assembled correctly, run this end-to-end verification sequence:

1. Stand Up the Test Environment

- Open **two separate tabs** (or one standard tab and one Incognito window) pointing to the User Emergency Portal: http://localhost:5000/index.html.
- Log in with different user accounts in each tab.
- Open a third separate browser window pointing to the Control Centre: http://localhost:5173/.

2. Synchronize GPS & Proximity

- On both user portal tabs, click the **Triangulate GPS** option. This records their coordinates into the backend' s active registry and starts a 15-second tracking loop. Ensure both users are simulated within a 5.0 km radius of one another.

3. Test Visual Evidence Uplink & SOS Dispatch

- In **User Tab A**, click the *Visual Situation Evidence Uplink* panel.
- Select an emergency photo. Verify that a local thumbnail preview appears cleanly on the screen.
- Click 🚨 **SEND SOS**.

4. Verify Real-Time System Reaction

- **Operations Command Dashboard:** Inspect the screen. It should instantly play an emergency chime alert, update the Recharts analytics graphs, and pin a glowing hazard marker on the Leaflet map. Click the **VIEW PHOTO** button or tap the map marker to view the uploaded Base64 image snapshot.
- **User Tab B (Nearby Citizen):** Check the screen of the neighbouring user. Because they are within the 5 km threat sector, they should automatically receive a flashing ⚠ NEARBY EMERGENCY WARNING banner toast and see the pulsing incident location pinned on their local map.

Tech stack (Tools used):-

To make this platform fast, reliable, and capable of real-time communication, we split our system into three main pieces: the **Frontend (what users see)**, the **Backend (the brains)**, and **Real-Time Communication (the nervous system)**.

The Frontend (The User Interfaces)

This is what citizens use to send for help and what operators use to manage rescues.

- **HTML (Hypertext Markup Language):** * *What it does:* The blueprint and skeleton of the app.
    - *Why we used it:* It builds the literal structure of the User Emergency Portal—the buttons, the text entry fields, the map containers, and the image upload boxes.
- **CSS & Tailwind CSS:** * *What it does:* The styling, layout, and visual theme.
    - *Why we used it:* It takes standard web pages and transforms them into high-contrast, clean, dark-neon interfaces. It ensures that text is easy to read during a high-stress crisis, layouts are responsive on phones, and critical buttons (like 🚨 SEND SOS) stand out instantly.
- **JavaScript (JS) & React.js:** * *What it does:* The interactivity and logic on the screen.
    - *Why we used it:* It handles all the action on the screen without forcing the page to constantly reload. It grabs the user's live GPS coordinates, converts uploaded disaster photos into lightweight text codes (Base64), and renders the interactive, glowing **Radar Map** and live **Analytics Graphs**.

The Backend (The Central Brain)

This runs in the background, processes data, handles security, and runs the math.

- **Node.js:** * *What it does:* The runtime environment that executes server-side code.
    - *Why we used it:* Node.js is incredibly fast at handling multiple data requests simultaneously. When hundreds of users are accessing the app at the exact same time during a localized disaster, Node.js keeps the backend running smoothly without crashing under heavy traffic.
- **Express.js:** * *What it does:* The backend web framework.
    - *Why we used it:* It maps out our system's endpoints—handling user login, registration, and managing the core business logic of our application.

Real-Time & Data Management Layer

This allows the frontend and backend to talk to each other instantly.

- **Socket.IO (WebSocket's):** * *What it does:* Instant, two-way, live data communication.
    - *Why we used it:* Standard internet connections require you to refresh a page to see new data. Socket.IO bypasses this completely. The second a user clicks "Send SOS," Socket.IO shoots the data to the server, calculates if other citizens are within **5.0 km** using the Haversine formula, and instantly

triggers the siren chimes and map pins on everyone else's screens.

- **Zero-Config JSON Database (database.js):** * *What it does:* Local data persistence.
    - *Why we used it:* For easy local development and testing, it saves active emergency tickets, safe shelters, and team statuses into local JSON files automatically. This ensures the project has zero setup friction for new developers while remaining ready to scale to heavy cloud databases (like MongoDB) later.

Data Flow Summary (How They Work Together)

[Citizen Interface] ──(HTML/CSS/JS) ──► Uploads Photo & Clicks SOS

(Socket.IO instantly transmits Base64 text string)



[Backend Server] ────(Node.js/Express) ──► 1. Saves ticket to Database



2. Runs 5km Proximity Check

├─► (Socket.IO Broadcast) ──► Pushes Warning Alerts to Nearby Citizens

Architecture summary: -

Our platform uses a **Decentralized Multi-Application Architecture**. Instead of building one massive, clunky application, we split the project into three distinct, independent modules that work together like a team.

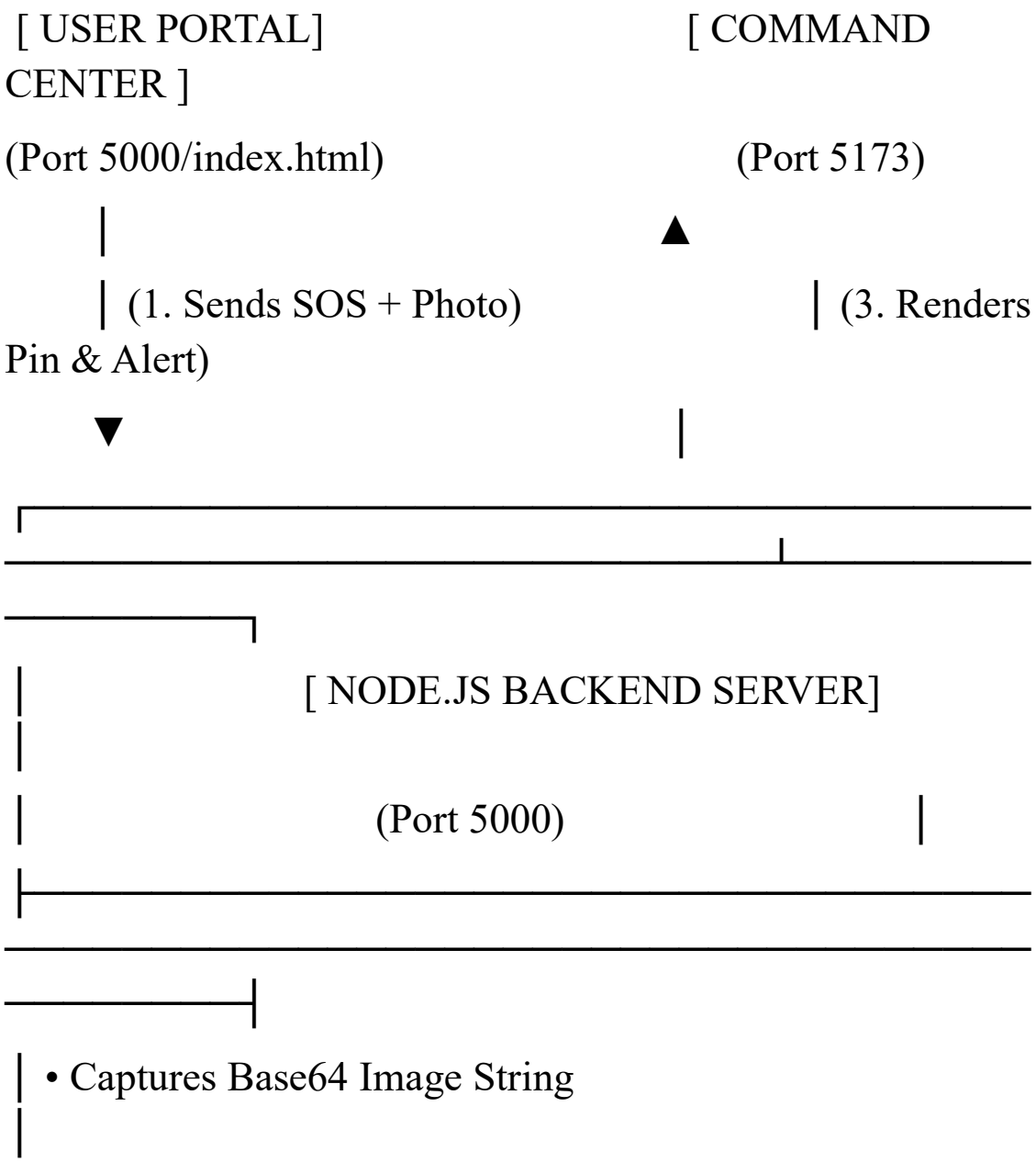
1. **The Directory Breakdown**

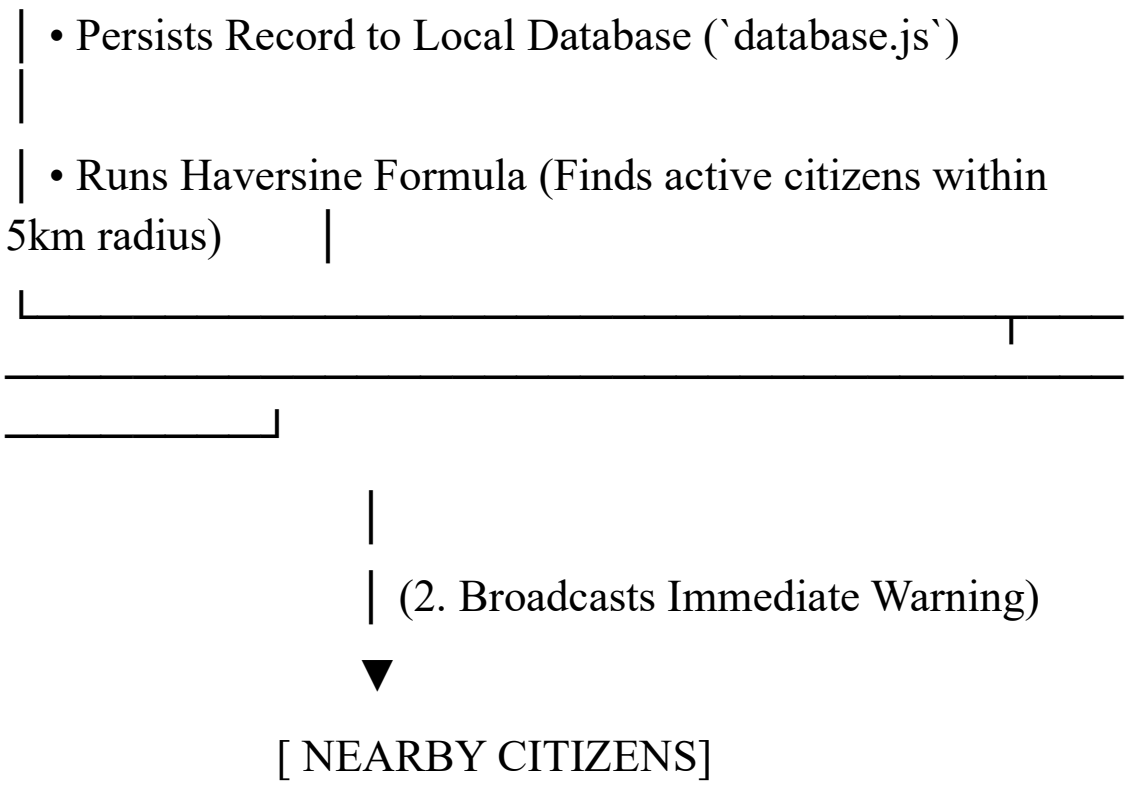
- **/User-app (The Citizen Portal):** A lightweight, browser-based interface built with HTML, CSS, and vanilla JavaScript. It is designed to run smoothly on any smartphone or computer web browser, allowing citizens to send distress signals, track rescue teams, and view nearby dangers.
- **/Backend (The Central Command Engine):** Built with Node.js and Express. This is the hidden power core of the system. It handles user authentication, tracks who is currently online, saves records, and calculates coordinate spaces.
- **/Controller-app (Operations Control Center):** A highly advanced React and Tailwind CSS application dashboard. This is the command room interface utilized by emergency managers to monitor entire cities, read AI-

prioritized lists, view uploaded live evidence, and route rescue teams.

2. The Real-Time Network Pipeline (How Data Moves)

Traditional apps wait for you to refresh the page to show new data. Our platform operates on a **Continuous Event Loop** powered by **Socket.IO**. Think of it as an open, instant phone line that stays connected 24/7.





3. Core Architectural Sub-Systems

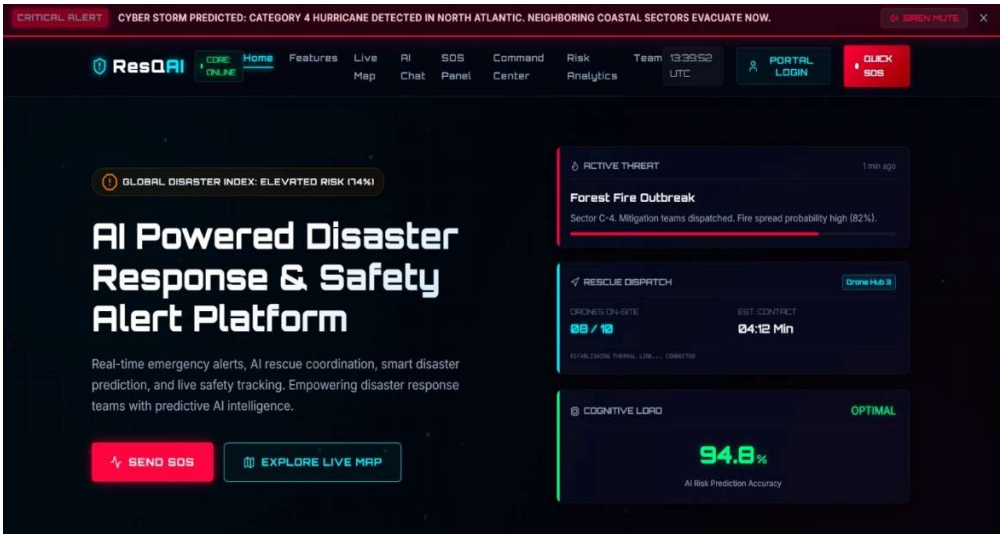
Automated Proximity Triangulation

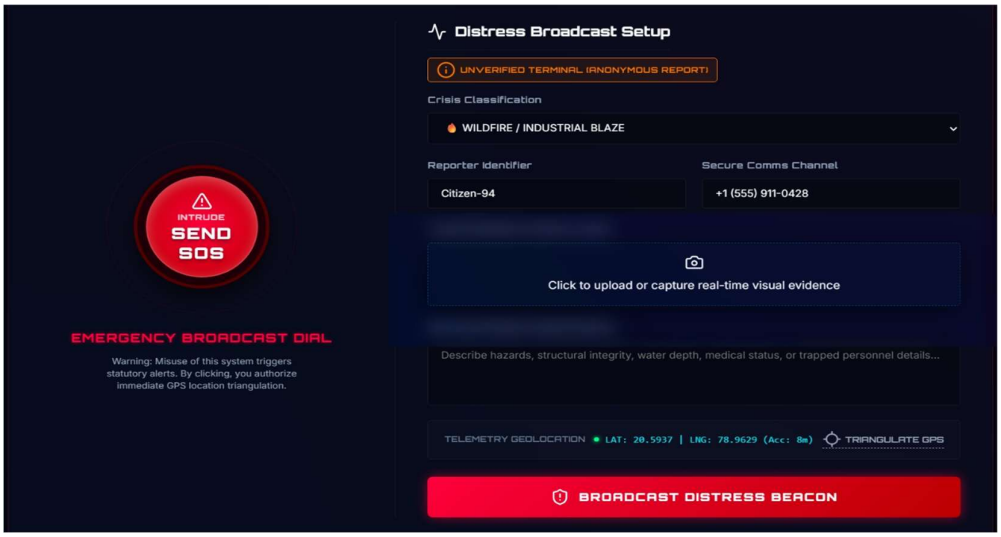
When a user logs in, a set Interval loop in app.js runs silently in the background every 15 seconds. It captures the device's GPS hardware location and synchronized updates back to the backend registry (active Citizens). If an SOS is fired, the server runs the **Haversine Formula** over this registry to calculate exactly who is in immediate danger.

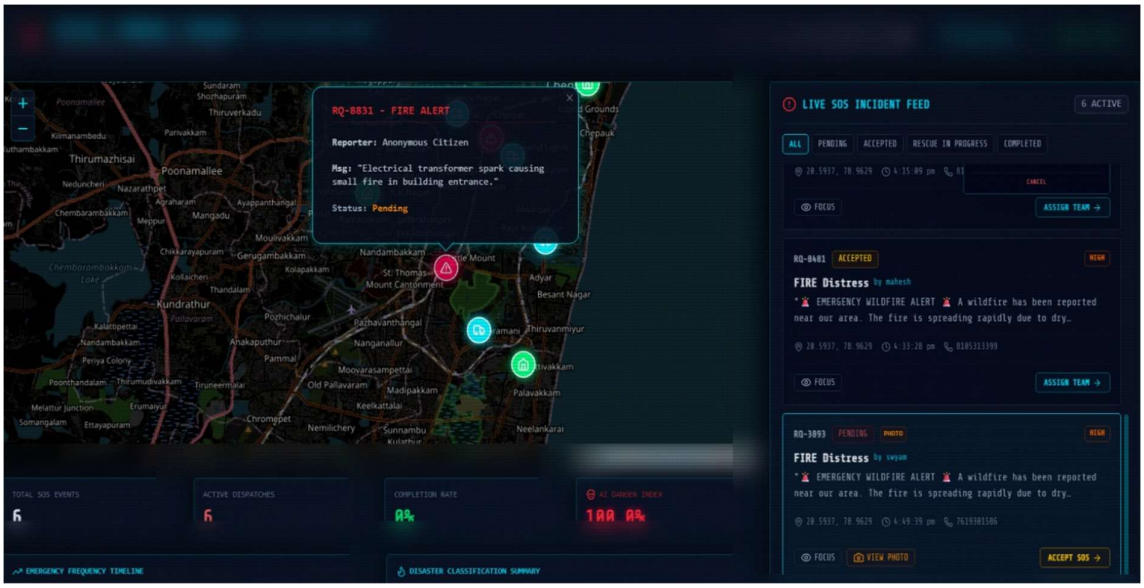
Lightweight Image Streaming Pipeline

Instead of relying on heavy, slow cloud file servers which can lag during a crisis, our frontend takes an uploaded photo and uses a File Reader component to encode the image instantly into a **Base64 text string**.

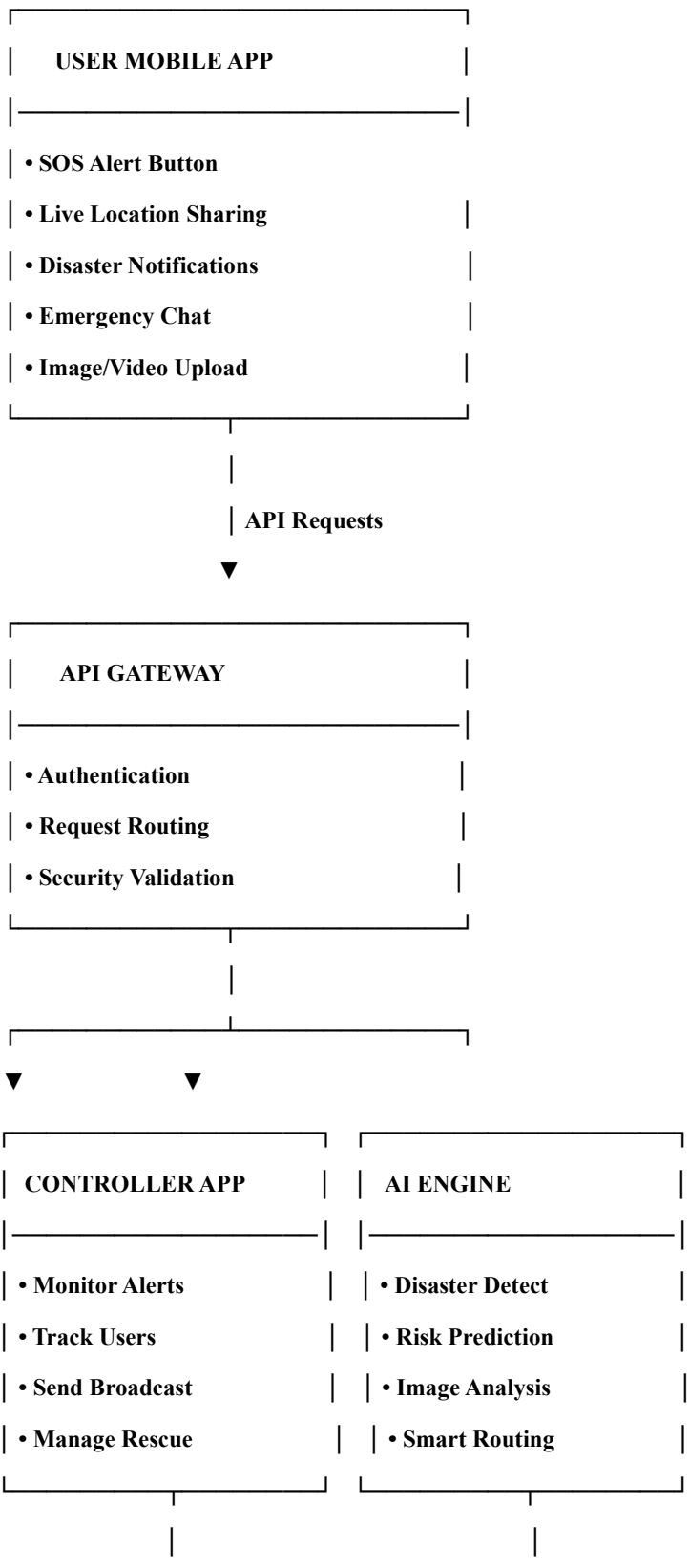
Demo:-

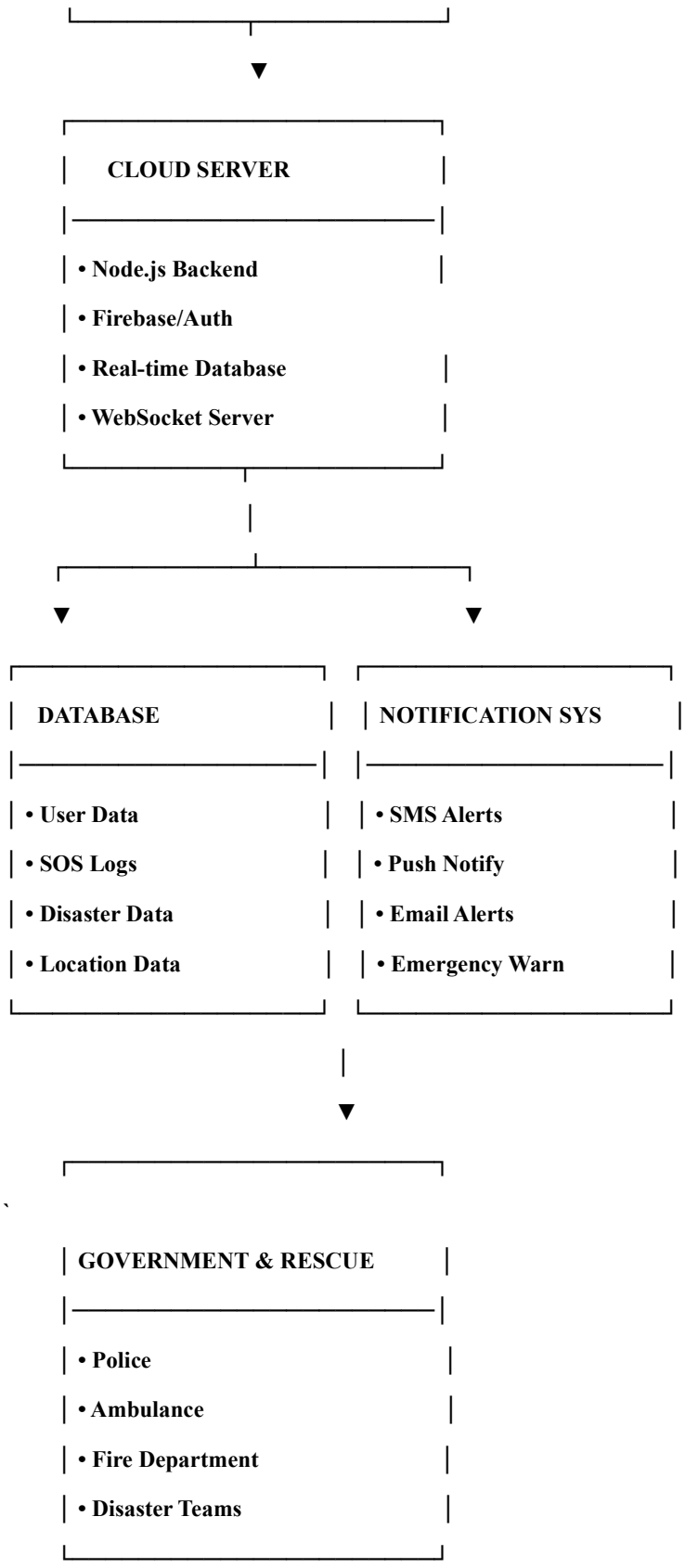

CRITICAL ALERT
CYBER STORM PREDICTED: CATEGORY 4 HURRICANE DETECTED IN NORTH ATLANTIC. NEIGHBORING COASTAL SECTORS EVACUATE NOW.
SIREN MUTE
ResQAI
CORE ONLINE
Home
Features
Live Map
AI Chat
SOS Panel
Command Center
Risk Analytics
Team
13:39:52 UTC
PORTAL LOGIN
QUICK SOS
GLOBAL DISASTER INDEX: ELEVATED RISK (74%)
AI Powered Disaster Response & Safety Alert Platform
Real-time emergency alerts, AI rescue coordination, smart disaster prediction, and live safety tracking. Empowering disaster response teams with predictive AI intelligence.
SEND SOS
EXPLORE LIVE MAP
ACTIVE THREAT
1 min ago
Forest Fire Outbreak
Sector C-4. Mitigation teams dispatched. Fire spread probability high (82%).
RESCUE DISPATCH
Drone Hub 3
DRONES ON-SITE
08 / 10
EST. CONTACT
04:12 Min
COGNITIVE LOAD
OPTIMAL
94.8%
AI Risk Prediction Accuracy


Distress Broadcast Setup
UNVERIFIED TERMINAL (ANONYMOUS REPORT)
Crisis Classification
WILDFIRE / INDUSTRIAL BLAZE
Reporter Identifier
Citizen-94
Secure Comms Channel
+1 (555) 911-0428
Click to upload or capture real-time visual evidence
Describe hazards, structural integrity, water depth, medical status, or trapped personnel details...
TELEMETRY GEOLOCATION
LAT: 20.5937 | LNG: 78.9629 (Acc: 8m)
TRIANGULATE GPS
BROADCAST DISTRESS BEACON
INTRUDE
SEND SOS
EMERGENCY BROADCAST DIAL
Warning: Misuse of this system triggers statutory alerts. By clicking, you authorize immediate GPS location triangulation.


RQ-8831 - FIRE ALERT
Reporter: Anonymous Citizen
Msg: "Electrical transformer spark causing small fire in building entrance."
Status: Pending
LIVE SOS INCIDENT FEED
6 ACTIVE
ALL
PENDING
ACCEPTED
RESCUE IN PROGRESS
COMPLETED
FOCUS
ASSIGN TEAM →
RQ-0401
ACCEPTED
HIGH
FIRE Distress by mahesh
" EMERGENCY WILDFIRE ALERT A wildfire has been reported near our area. The fire is spreading rapidly due to dry…
FOCUS
ASSIGN TEAM →
RQ-3093
PENDING
PHOTO
HIGH
FIRE Distress by swyam
" EMERGENCY WILDFIRE ALERT A wildfire has been reported near our area. The fire is spreading rapidly due to dry…
FOCUS
VIEW PHOTO
ACCEPT SOS →
TOTAL SOS EVENTS
ACTIVE DISPATCHES
COMPLETION RATE
AI DARKER INDEX
EMERGENCY FREQUENCY TIMELINE
DISASTER CLASSIFICATION SUMMARY

Architecture Diagram:-




CLOUD SERVER
• Node.js Backend
• Firebase/Auth
• Real-time Database
• WebSocket Server
DATABASE
• User Data
• SOS Logs
• Disaster Data
• Location Data
NOTIFICATION SYS
• SMS Alerts
• Push Notify
• Email Alerts
• Emergency Warn
GOVERNMENT & RESCUE
• Police
• Ambulance
• Fire Department
• Disaster Teams